

Architectural Paradigms for High-Fluidity Industrial HMI Visualization: A Technical Analysis of Open-Source Frameworks and Real-Time Rendering Engines

The transition from traditional supervisory control and data acquisition (SCADA) systems to modern Industrial Internet of Things (IIoT) ecosystems has fundamentally altered the requirements for human-machine interface (HMI) design. In the current industrial landscape, operators and engineers no longer rely on static or low-refresh-rate displays; instead, they require high-fluidity visualization platforms capable of processing massive data throughput with minimal latency.¹ High fluidity, generally defined as maintaining a consistent update rate of 60 frames per second (FPS) or higher, is essential for reducing operator fatigue and ensuring that time-critical anomalies are identified instantaneously.¹ This report provides an exhaustive technical analysis of open-source frameworks prioritized for 2D and 3D visualization, high-throughput charting, and the integration of digital twins into industrial workflows.

The Computational Foundations of Visual Fluidity

Achieving high fluidity in an IIoT context is a multi-dimensional challenge involving hardware-accelerated rendering, efficient data serialization, and optimized client-side processing. The fundamental constraint for any fluid interface is the render budget, mathematically expressed by the relationship between the desired frame rate and the time available to process each frame. To achieve a refresh rate of 60 FPS, the system must complete all telemetry ingestion, state updates, layout calculations, and pixel rasterization within a strict temporal window:

$$T_{\text{frame}} = \frac{1}{60} \approx 16.67 \text{ milliseconds}$$

In high-throughput environments where data points arrive at kilohertz frequencies, this budget is often exceeded by the overhead of traditional web technologies such as the Document Object Model (DOM).⁴ Consequently, the industry has seen a shift toward "Spartan" rendering technologies—specifically Canvas 2D, WebGL, and native machine-code compilation—that bypass the high-level abstractions of general-purpose web development.¹

The choice of rendering technology dictates the maximum achievable data density. While Scalable Vector Graphics (SVG) are excellent for high-fidelity, resolution-independent widgets, they struggle when thousands of interactive elements are present due to the CPU-intensive task of managing the DOM tree.¹ Conversely, Canvas 2D and WebGL offload the rasterization

process to the Graphics Processing Unit (GPU), enabling the visualization of millions of data points without dropping below the 60 FPS threshold.⁵

High-Fluidity 2D Visualization Frameworks

In the 2D domain, the priority is on responsive, touch-optimized interfaces that can run on hardware ranging from low-power microcontrollers (MCUs) to advanced industrial PCs. Open-source frameworks in this category are evaluated based on their rendering efficiency, memory safety, and cross-platform portability.

Slint: The Performance Benchmark for Embedded Systems

Slint, formerly SixtyFPS, represents a paradigm shift in embedded UI development. Developed in Rust to leverage its inherent memory safety and performance characteristics, Slint is designed specifically to deliver 60 FPS graphics on modest hardware.¹ Its architecture uses a declarative domain-specific language (DSL) to describe the UI, which is then compiled directly into native C++ or Rust code.¹ This compilation step is critical because it removes the need for a heavy runtime interpreter or a virtual machine, resulting in binaries that are exceptionally small and fast to initialize.

Slint's runtime requirements are minimal, often fitting in less than 300 KiB of RAM, which makes it suitable for cost-effective hardware such as those running low-powered processors on bare metal or Linux.¹ For industrial panels, Slint supports multiple processor architectures and rendering backends, including GPU acceleration, DMA2D for STM32-based systems, and direct framebuffers for devices without dedicated GPUs.¹ Its reactive property system ensures that the UI only re-renders the specific components affected by telemetry updates, thereby preserving CPU cycles for critical control logic.¹

Flutter: Custom Rendering for Visual Consistency

Google's Flutter has emerged as a major contender for high-end industrial HMIs due to its unique architectural approach. Unlike other cross-platform frameworks that use native system controls, Flutter draws every pixel of its interface using its own high-performance rendering engine, Skia (now transitioning to Impeller).¹ This ensures that the HMI looks and behaves identically across diverse hardware, from Android tablets to embedded Linux panels.¹

The fluidity of Flutter is attributed to its C/C++ core, which handles text layout, graphics, and animations at the engine level, while the UI logic resides in a high-productivity Dart layer.¹ For industrial applications requiring branded, highly animated interfaces, Flutter provides the flexibility to create custom widgets that respond to touch and data inputs with 60–120 FPS fluidity, provided hardware acceleration is available.¹ Furthermore, Flutter's expansion into the desktop and web ecosystems allows for a "write once, deploy everywhere" strategy that is increasingly attractive to original equipment manufacturers (OEMs).¹

LVGL: The Open-Source Standard for Resource-Constrained Devices

The Light and Versatile Graphics Library (LVGL) is the predominant choice for HMI on

microcontrollers.¹¹ It is an open-source, hardware-agnostic C library that provides a comprehensive toolkit for building modern GUIs on devices with limited memory and processing power.¹¹ LVGL's modular design allows developers to enable only the widgets and features required for a specific application, thereby optimizing the flash and RAM footprint.¹² One of LVGL's primary advantages in industrial settings is its support for hardware acceleration. It provides APIs that allow drawing operations—such as rectangle fills, image transformations, and anti-aliasing—to be offloaded to dedicated hardware units like the Renesas RA or RZ series accelerators.¹¹ This ensures that the interface remains responsive even when the CPU is busy with networking or security tasks.¹¹ LVGL also supports multi-touch gestures, smooth scrolling, and complex animations, bringing a consumer-grade user experience to industrial thermostats, medical devices, and smart home appliances.¹²

Comparative Table of 2D HMI Frameworks

Framework	Primary Language	Rendering Backend	Typical RAM Usage	License
Slint	Rust / C++	Native GPU / Framebuffer	< 300 KiB	GPLv3 / Apache 2.0
LVGL	C / C++	Software / HW Accelerated	4 KB - 48 KB	MIT
Flutter	Dart / C++	Impeller / Skia	20 MB - 100 MB	BSD
Qt Quick	QML / C++	Scenegraph / OpenGL	10 MB - 50 MB	LGPL / Commercial
React Native	JavaScript	Native UI Bridge	30 MB - 150 MB	MIT

1

High-Throughput Charting and Real-Time Waveform Visualization

In the IIoT sector, the ability to visualize high-frequency time-series data is critical for condition monitoring, predictive maintenance, and quality control. High throughput refers to the library's capacity to ingest, process, and render thousands or even millions of data points per second without compromising the responsiveness of the overall HMI.⁵

uPlot: The Apex of Efficiency for Time-Series Data

For applications where the primary goal is the fluid visualization of dense time-series data, uPlot is the benchmark library.⁴ It is a Canvas 2D-based chart library designed with an intense focus on minimalism and performance.⁴ From a cold start, uPlot can create an interactive chart containing 150,000 to 166,000 data points in approximately 25 to 135 milliseconds, depending on the hardware.⁶

The library’s performance is attributed to its avoidance of DOM overhead and its linear scaling at approximately 100,000 points per millisecond during updates.⁶ For industrial real-time monitoring, uPlot can live-stream data at 60 FPS, maintaining its efficiency up to approximately 100,000 in-view points.⁶ Beyond this, it is recommended to employ data decimation or move to a WebGL-based solution.⁵ uPlot is particularly favored by "hacker" developers and industrial engineers who value a tiny bundle size (under 50 KB) over flashy animations and flourishes.⁴

webgl-plot: Native GPU Waveform Rendering

When the data throughput requirements exceed the capabilities of Canvas 2D, webgl-plot provides a high-performance solution using native WebGL.¹⁸ This library is specifically designed for real-time plotting of multiple waveforms, such as those produced by software-based oscilloscopes, FPGAs, or high-speed microcontrollers.¹⁸ By utilizing the GPU for line drawing, webgl-plot achieves update rates that match the screen's refresh rate even with extremely large datasets.¹⁸

The library supports both thin (native) and thick lines, though thick lines are computationally more expensive and should be used judiciously.¹⁸ Its compatibility with OffScreenCanvas and WebWorkers is a significant advantage for industrial HMI, as it allows the rendering logic to be offloaded from the main thread, ensuring that the user interface remains responsive to interactions even during heavy data ingestion.¹⁸ For embedded systems connected via WebUSB, Web Bluetooth, or Serial APIs, webgl-plot offers a direct path to low-latency visualization.¹⁸

Apache ECharts: Versatile Enterprise-Grade Visualization

Apache ECharts is a comprehensive, open-source data visualization platform that has gained significant traction in industrial dashboards due to its versatility and extensive feature set.¹⁶ While it is heavier than uPlot, it offers a wider array of chart types—including 3D scatter plots, geospatial maps, and complex heatmaps—making it ideal for enterprise-level IIoT platforms.¹⁶ ECharts supports both Canvas and SVG rendering, providing developers with the flexibility to choose the best technology for their specific use case.²² For high-throughput charts, its Canvas renderer is capable of handling large datasets, though its performance benchmarks typically fall behind those of uPlot and webgl-plot in terms of raw update speed and CPU utilization.⁶ Nevertheless, its professional look, deep customization options, and official wrappers for frameworks like React, Vue, and Angular make it a highly productive choice for industrial developers.²¹

Time-Series Visualization Performance Comparison

Library	Rendering Tech	Millions of Points Support	60 FPS Streaming	Typical Bundle Size
uPlot	Canvas 2D	Yes (Binned)	Yes (<100k pts)	~40-50 KB
webgl-plot	WebGL	Yes	Yes (Waveforms)	< 100 KB
Apache ECharts	Canvas / SVG	Moderate	Variable	~1 MB

Chart.js	Canvas 2D	No	Limited	~48 KB
Highcharts	SVG	Moderate	Limited	~200-500 KB

4

3D Visualization and Industrial Digital Twins

The integration of 3D visualization into IIoT HMIs allows for a more intuitive representation of complex physical systems, enabling operators to visualize machine states, temperature gradients, and mechanical collisions in a spatial context.²⁴ Digital twins—3D replicas of real-world systems linked to real-time sensor data—are increasingly used for predictive analytics and virtual commissioning.²⁵

Babylon.js: The Premier Engine for Industrial WebXR

Babylon.js is a fully featured, open-source 3D rendering engine built on JavaScript and WebGL, making it a powerful choice for creating digital twin applications that run directly in a web browser.²⁶ Its architecture is designed for ease of use while providing high-end features such as physically based rendering (PBR), particle systems, and skeletal animations.²⁹

In an industrial HMI, Babylon.js can be used to visualize real-time data by dynamically changing the colors, materials, or animations of 3D objects based on sensor telemetry.²⁶ The engine's support for the glTF format—often called the "JPEG for 3D"—ensures that models from CAD software can be imported seamlessly while retaining consistent visual quality.²⁹ Furthermore, the "Babylon React Native" project allows developers to bring these interactive 3D experiences to mobile platforms with native-level performance, bridging the gap between web and handheld industrial tools.²⁹

CesiumJS: Geospatial Visualization and Large-Scale Twins

For IIoT applications that span large geographical areas—such as smart cities, drone fleet monitoring, or energy distribution networks—CesiumJS is the industry standard.³⁴ It is an open-source JavaScript library for creating 3D globes and maps with high precision and performance.³⁴ CesiumJS utilizes open formats like 3D Tiles to stream massive geospatial datasets, including 3D buildings, photogrammetry, and point clouds.³⁴

CesiumJS provides first-class support for time-dynamic visualization, allowing developers to represent real-time telemetry from moving assets (like satellites or AGVs) directly on a 3D WGS84 globe.³⁴ Its synchronous 2D/3D display capabilities and support for diverse geometry types make it a versatile tool for digital twins that require both a high-level geographical overview and detailed localized views.³⁴

Open Cascade and CAD Integration

In manufacturing environments, the ability to visualize the actual CAD geometry of parts and assemblies is often required for precise maintenance and inspection.²⁴ Open Cascade provides

an open-source CAD kernel and a set of advanced visualization technologies for managing complex 3D data.²⁵ Their digital twin solution covers the entire lifecycle of an industrial asset, aggregating heterogeneous data from IoT sensors, PLM, and MES systems around a hybrid 3D model that can include both CAD data and point clouds.²⁵ This approach allows for advanced visual representation of moving parts, collisions, and thermal profiles in a way that is easily understandable by non-specialists.²⁵

Digital Twin Visualization Comparison

Tool / Library	Focus	Open Source License	WebGL Support	Key Strength
Babylon.js	General 3D / IoT	Apache 2.0	Yes	PBR, Animations, WebXR
CesiumJS	Geospatial / 3D Maps	Apache 2.0	Yes	3D Tiles, Global Precision
Three.js	General 3D	MIT	Yes	Ecosystem, Ease of Use
Open Cascade	CAD / Manufacturing	LGPL	Yes	CAD Kernel, Point Clouds
Gazebo	Robotics Cells	Apache 2.0	Yes	Physics, ROS Integration

25

Integrated Open-Source HMI and SCADA Platforms

While individual libraries provide the visualization building blocks, several integrated open-source platforms offer a complete solution for industrial monitoring, combining data acquisition (DAQ) with sophisticated dashboarding tools.

FUXA: Modern, Web-Based Industrial Visualization

FUXA is a comprehensive web-based SCADA/HMI platform designed for the IIoT era.² It allows users to build modern dashboards and connect to industrial devices using standard protocols such as Modbus RTU/TCP, Siemens S7, OPC-UA, and MQTT.⁷ FUXA's engineering and design environment is completely web-based, utilizing a visual editor that supports SVG widgets and interactive graphics.⁷

FUXA's frontend is built with Angular and HTML5, ensuring that dashboards are responsive across Linux, Windows, macOS, and Raspberry Pi.⁷ While it primarily focuses on 2D SVG-based visualization, its open-source nature (MIT license) and extensibility make it a strong candidate for industrial monitoring where a modern, web-first architecture is desired.⁷ Current maintainers have indicated that while native 3D support is not yet integrated, a future approach using Three.js or Babylon.js widgets to load glTF models is a prioritized feature request.³⁷

ThingsBoard Community Edition: The Scalable IoT Powerhouse

ThingsBoard CE is widely considered the most feature-rich open-source IoT platform for device management and data visualization.³⁸ It enables companies to build rich IoT dashboards with over 600 customizable widgets, supporting real-time data visualization and remote device control.³⁹ ThingsBoard's architecture is designed for high scalability and fault tolerance, capable of handling millions of devices in a cluster.³⁹

For HMI applications, ThingsBoard CE provides digital twin modeling, a powerful rule engine for data processing and alarms, and support for a vast array of connectivity protocols (MQTT, CoAP, HTTP, LwM2M, etc.).³⁸ While the Professional Edition adds business-specific features like white-labeling and advanced RBAC, the Community Edition remains a robust, 100% open-source tool for collecting, storing, and analyzing IoT data.³⁸

Node-RED Dashboard 2.0: Flexible and Flow-Based

The FlowFuse Dashboard, also known as Node-RED Dashboard 2.0, is the modern successor to the original Node-RED dashboard, built to overcome the limitations of the deprecated Angular 1.0 architecture.⁴² It provides a collection of nodes for creating data-driven applications and visualizations with a focus on ease of use and flexibility.⁴³

Dashboard 2.0 offers multiple layout options, including a 12-column grid structure, fixed-pixel layouts, and "notebook" layouts for analysis-heavy interfaces.⁴³ It supports multi-tenancy out of the box, allowing different users to see unique data based on their session.⁴³ Through the ui-template node, developers can integrate custom HTML, JavaScript, and even Vue components to create highly specialized or high-fluidity visualizations that go beyond the standard widget set.⁴³

Open-Source HMI/SCADA Feature Matrix

Platform	License	Backend	Frontend	Protocol Drivers
FUXA	MIT	Node.js	Angular	Modbus, S7, OPC-UA, MQTT
ThingsBoard CE	Apache 2.0	Java / Spring	Angular	MQTT, CoAP, HTTP, LwM2M
Node-RED D2	Apache 2.0	Node.js	Vue.js	MQTT, Modbus, REST, etc.
ScadaBR	GPL	Java	HTML / JSP	Modbus, BACnet, DNP3
AdvancedHMI	Free/Prop	.NET	WinForms	Allen Bradley, Modbus, DF1

Strategic Optimization for High-Throughput Pipelines

Even the most performant visualization framework will stutter if the underlying data pipeline is not optimized. Industrial charting failures often occur at four distinct layers: backend data volume, payload serialization, client-side memory management, and rendering execution.⁵

Architectural Latency Budgets

To maintain 60 FPS, the end-to-end latency budget must be meticulously managed. Streaming systems like Kafka and ClickHouse can achieve ingestion-to-queryable latencies of 50–100ms.⁵ Optimized columnar databases can execute aggregations in 10–50ms.⁵ However, if the HMI attempts to download and parse a massive JSON payload every few seconds, the client-side parsing will block the main thread, leading to visual lag.

The recommended strategy is to use binary serialization formats like Protobuf or specialized TypedArrays (e.g., Float32Array).⁵ These formats are compact—approximately 80 KB for 10,000 points compared to 500 KB for JSON—and require zero parsing, as they are directly usable by the browser's memory.⁵

Intelligent Data Decimation: The LTTB Algorithm

When displaying historical trends over long periods, rendering every single data point is both computationally wasteful and visually counterproductive. The Largest Triangle Three Buckets (LTTB) algorithm is designed specifically for time-series visualization.⁵ It divides data into buckets based on the target number of pixels on the screen and selects points that preserve the visual "character" (peaks and valleys) of the original signal.⁵ By automatically selecting the appropriate resolution based on the time range—for example, using 1-second intervals for a 1-hour range and 1-hour intervals for a 30-day range—the HMI can maintain high fluidity during zooming and panning operations.⁵

Implementation Considerations for Professional HMI

When selecting a framework or library for an industrial project, several qualitative factors must be considered alongside raw performance benchmarks. These include the maturity of the documentation, the robustness of the community, and the ease of integration with existing automation infrastructure.

Documentation and Ecosystem Support

Frameworks like Apache ECharts and Highcharts are favored in enterprise environments because they provide extremely complete documentation, searchable examples, and professional support options.⁴ In contrast, high-performance "hacker" tools like uPlot often have minimal documentation, requiring developers to reverse-engineer examples to understand complex APIs.⁴ For mission-critical HMIs where long-term maintenance is a concern, the "safe" choice is often a framework with an established ecosystem and commercial backing, such as Qt or ThingsBoard.¹

Hardware and OS Portability

The trend toward containerization (Docker) and web technologies has made cross-platform portability easier, but specific industrial hardware still presents challenges. Slint and LVGL are uniquely suited for microcontrollers and bare-metal environments where a full Linux OS is not available.¹ For high-end panels, platforms like FUXA or Node-RED can be packaged as standalone desktop applications using Electron, providing a native-like experience while leveraging web-based visualization fluidity.⁷

Security and Multi-Tenancy

In modern IIoT environments, security is no longer an afterthought. Open-source platforms like ThingsBoard and FUXA include built-in support for role-based access control (RBAC), multi-tenancy, and secure communications (SSL/TLS).⁷ These features are essential when the HMI is accessed by multiple stakeholders across different regions or when it control mission-critical infrastructure.⁴⁷

Conclusion: Synthesis of High-Fluidity Visualization Trends

The landscape of open-source IIoT HMI visualization in 2025 is defined by a clear separation between lightweight embedded frameworks and high-performance web-based analytics tools. For the highest possible fluidity in 2D interfaces, **Slint** and **LVGL** provide the necessary proximity to hardware for MCUs and MPUs.¹ For web-based dashboards, **Flutter** and optimized JavaScript platforms like **Node-RED Dashboard 2.0** offer a balance of visual fidelity and rapid development.¹

In the realm of high-throughput data, **uPlot** remains the unrivaled leader for Canvas 2D time-series visualization, while **webgl-plot** serves as the optimal choice for GPU-accelerated waveforms.⁶ 3D digital twins are best served by the robust ecosystems of **Babylon.js** for localized asset monitoring and **CesiumJS** for large-scale geospatial situational awareness.²⁶ Ultimately, the choice of a visualization stack must be integrated into a broader architectural strategy that prioritizes binary data transfer, intelligent decimation, and a strict frame budget. By leveraging the specific strengths of these open-source tools, industrial organizations can build HMI systems that are not only aesthetically modern but also technically capable of meeting the extreme demands of the IIoT era.

Works cited

1. Top 5 Embedded HMI UI Frameworks for 2025 - Spyrosoft, accessed March 16, 2026, <https://spyro-soft.com/blog/hmi/embedded-ui-frameworks>
2. Best HMI Software 2025 | Expert Reviews & Comparison - PLC Programming, accessed March 16, 2026, <https://plcprogramming.io/blog/best-hmi-software-2025>

3. Slint | Declarative GUI for Rust, C++, JavaScript & Python, accessed March 16, 2026, <https://slint.dev/>
4. My Thoughts on the uPlot Charting Library - Casey Primozic, accessed March 16, 2026, <https://cprimozic.net/notes/posts/my-thoughts-on-the-uplot-charting-library/>
5. Fast Charts with Real-Time Data: Fix Slow Dashboards Fast - Tinybird, accessed March 16, 2026, <https://www.tinybird.co/blog/Fast-Charts-real-Time-Data>
6. GitHub - leeoniya/uPlot: A small, fast chart for time series, lines, areas, ohlc & bars, accessed March 16, 2026, <https://github.com/leeoniya/uPlot>
7. frangoteam/FUXA: Web-based Process Visualization ... - GitHub, accessed March 16, 2026, <https://github.com/frangoteam/FUXA>
8. WebGL charting libraries - Reddit, accessed March 16, 2026, https://www.reddit.com/r/webgl/comments/qg96r8/webgl_charting_libraries/
9. Enhance UI with Slint Development by Witekio, accessed March 16, 2026, <https://witekio.com/embedded-gui/c-application-development/slint/>
10. Cross Platform - Slint, accessed March 16, 2026, <https://slint.dev/cross-platform>
11. Create a Cost-Effective HMI With LVGL (and Some Help From Renesas), accessed March 16, 2026, <https://embeddedcomputing.com/technology/open-source/create-a-cost-effective-hmi-with-lvgl-and-some-help-from-renesas>
12. LVGL: Redefining HMI Applications with Lightweight and Versatile Graphics | by elnfochips (An Arrow Company) | Medium, accessed March 16, 2026, <https://medium.com/@elnfochips/lvgl-redefining-hmi-applications-with-lightweight-and-versatile-graphics-9ae6b126a69e>
13. Features - LVGL, accessed March 16, 2026, <https://lvgl.io/features>
14. LVGL: Lightweight GUI for Embedded HMI Applications - elnfochips, accessed March 16, 2026, <https://www.elnfochips.com/blog/light-and-versatile-graphics-library-lvgl-excellence-for-hmi-application/>
15. 5 Best JS Chart Libraries for Data Visualization in 2026 | Qrvey, accessed March 16, 2026, <https://qrvey.com/blog/js-chart-library/>
16. GitHub - hal9ai/awesome-dataviz: :chart_with_upwards_trend: A curated list of awesome data visualization libraries and resources., accessed March 16, 2026, <https://github.com/hal9ai/awesome-dataviz>
17. npm:uplot - Skypack.dev, accessed March 16, 2026, <https://www.skypack.dev/view/uplot>
18. webgl-plot - NPM, accessed March 16, 2026, <https://www.npmjs.com/package/webgl-plot>
19. danchitnis/webgl-plot: A high-Performance real-time 2D ... - GitHub, accessed March 16, 2026, <https://github.com/danchitnis/webgl-plot>
20. 10 Best Open Source & Free BI Tools for 2025 - Domo, accessed March 16, 2026, <https://www.domo.com/learn/article/open-source-bi-tools>
21. Comparing 8 Popular React Charting Libraries — Performance, Features, and Use Cases, accessed March 16, 2026, <https://medium.com/@ponshriharini/comparing-8-popular-react-charting-libraries>

- [s-performance-features-and-use-cases-cc178d80b3ba](#)
22. Business Charts plugin for Grafana | Grafana Labs, accessed March 16, 2026, <https://grafana.com/grafana/plugins/volkovlabs-echarts-panel/>
 23. Top 5 Chart Libraries to use in Your Next Project - Strapi, accessed March 16, 2026, <https://strapi.io/blog/chart-libraries>
 24. Human-Machine Interface Visualization - 3DViewStation, accessed March 16, 2026, <https://www.3dviewstation.com/integrations/visualization-for-hmi-on-machines.html>
 25. 3D digital twin solution - Open Cascade, accessed March 16, 2026, https://www.opencascade.com/use_cases/digital-twin-solution/
 26. Babylon.js Digital Twins and IoT, accessed March 16, 2026, <https://www.babylonjs.com/digitalTwinIot/>
 27. Top Free & Open Source Digital Twin and Plant Simulation Tools - MDCplus, accessed March 16, 2026, <https://mdcplus.fi/blog/top-free-digital-twin-plant-simulation-manufacturing/>
 28. Digital Twins in the Machinery & Robotics Industry | Unity, accessed March 16, 2026, <https://unity.com/blog/digital-twins-machinery-robotics-industry>
 29. Babylon Native: cross-platform native apps with the power of Babylon.js, accessed March 16, 2026, <https://www.babylonjs.com/native/>
 30. BabylonJS/BabylonNative: Build cross-platform native applications with the power of the Babylon.js JavaScript framework - GitHub, accessed March 16, 2026, <https://github.com/BabylonJS/BabylonNative>
 31. NEM_2024_HMI_B/guide.md at master - GitHub, accessed March 16, 2026, https://github.com/bstiffler582/NEM_2024_HMI_B/blob/master/guide.md
 32. glTF (GL Transmission Format) - Sketchfab, accessed March 16, 2026, <https://sketchfab.com/features/gltf>
 33. Reactylon: An open-source framework for building cross-platform WebXR apps with React + Babylon.js : r/reactjs - Reddit, accessed March 16, 2026, https://www.reddit.com/r/reactjs/comments/1kf8cvy/reactylon_an_opensource_framework_for_building/
 34. CesiumJS - Cesium, accessed March 16, 2026, <https://cesium.com/platform/cesiumjs/>
 35. Visualizations for the digital twin - Fraunhofer-Gesellschaft, accessed March 16, 2026, <https://www.igd.fraunhofer.de/en/industries/infrastructure-and-public-services/infrastructure/visualizations-for-the-digital-twin.html>
 36. lucas-bremond/satellite-visualizer: Grafana panel plugin to visualize a satellite in 3D using CesiumJS. - GitHub, accessed March 16, 2026, <https://github.com/lucas-bremond/satellite-visualizer>
 37. Support for Importing 3D Scenes (.glb) in FUXA #2148 - GitHub, accessed March 16, 2026, <https://github.com/frangoteam/FUXA/discussions/2148>
 38. ThingsBoard Community vs Professional Edition: Which Fits Your IoT Project? - WZ-IT, accessed March 16, 2026, <https://wz-it.com/en/blog/thingsboard-community-vs-professional-edition-comp>

[arison/](#)

39. ThingsBoard Community Edition vs Professional Edition: What's the Difference, accessed March 16, 2026, <https://thingsboard.io/ce-vs-pe-diff/>
40. ThingsBoard — Open-source IoT (Internet of Things) Platform, accessed March 16, 2026, <https://thingsboard.io/>
41. Frequently Asked Questions | ThingsBoard Community Edition, accessed March 16, 2026, <https://thingsboard.io/docs/faq/>
42. Dashboard and Dashboard 2.0 - Node-RED, accessed March 16, 2026, <https://discourse.nodered.org/t/dashboard-and-dashboard-2-0/90937>
43. Getting Started - Node-RED Dashboard 2.0 - FlowFuse, accessed March 16, 2026, <https://dashboard.flowfuse.com/getting-started.html>
44. Node-RED Dashboard 2.0, accessed March 16, 2026, <https://dashboard.flowfuse.com/>
45. Best Open Source HMI Software 2026 - SourceForge, accessed March 16, 2026, <https://sourceforge.net/directory/hmi/>
46. JavaScript charting library for web apps, dashboards, grids, React and more | Highcharts | Highcharts, accessed March 16, 2026, <https://www.highcharts.com/>
47. Top Affordable HMI Software & Solutions | Tatsoft, accessed March 16, 2026, <https://tatsoft.com/hmi-software/>
48. IoT Web HMI | Open Automation Software, accessed March 16, 2026, <https://openautomationsoftware.com/web-hmi/>
49. Frequently Asked Questions | ThingsBoard Professional Edition, accessed March 16, 2026, <https://thingsboard.io/docs/pe/faq/>